Thakur Educational Trust's (Regd.)
THAKUR COLLEGE OF SCIENCE & COMMERCE
Empowered Autonomous College Permanently Affiliated to University of Mumbai
(NAAC Accredited with Grade "A" (3rd Cycle) & ISO 21001:2018 Certified)
Best College Award by University of Mumbai for the Year 2018-2019

DATA SCIENCE JOURNAL

Submitted by
PRAGATI SHUKLA - 264428
2026-2027

Under the guidance of
Prof. Ms. RIMSY DUA

Submitted in partial fulfilment of requirement for qualifying MSc.-(IT)

Semester-V Examination

Thakur College of Science and Commerce,

Thakur Village, Kandivali(E),

Mumbai-400101

Thakur Educational Trust's (Regd.)
THAKUR COLLEGE OF SCIENCE & COMMERCE
Empowered Autonomous College Permanently Affiliated to University of Mumbai
(NAAC Accredited with Grade "A" (3rd Cycle) & ISO 21001:2018 Certified)
Best College Award by University of Mumbai for the Year 2018-2019

Certificate

This is to certify that the Data Science Journal**.** Undertaken at the "Thakur College of Science and Commerce" by **Pragati Shukla 264428** in partial fulfilment of MSc. IT degree Examination had not been submitted for our other examination and does not form part of any other course undergone by the candidate.

It is further certified that we have completed all required phases of the journal.

External Examiner

Internal Examiner

Project Guide

Head of the Department

DATA SCIENCE-INDEX PAGE

S.No.	Name of the Practical	Date	Signature
1a)	To extract data from an image and transform it into a structured, machine-readable format.	27-06 -2026	
b)	To process and prepare raw CSV data to conform to the "HORUS" standard, making it suitable for analysis.	27-06 -2026	
2a)	Removing Leading, Lagging or extra spaces from a data entry.	11-07-2026	
b)	Removing non-printable characters .	11-07-2026	
c)	Reformatting Data Entry.	11-07-2026	
d)	To detect outliers using a fixed latitude range.	11-07-2026	
e)	To calculate average latitude by country and place name.	11-07-2026	
f)	To visualize data binning with a histogram and normal distribution fit.	18-07-2026	
3	To understand the use of retrieve superstep.	18-07-2026	
4	To understand the use of histogram,pie chart and line chart in data science.	18-07-2026	
5	To implement complex data visualizations.	25-07-2026	
6	To understand the use of Assess superstep.	1-08-2026	
7	To implement line detection using hough transform.	8-08-2026	
8	To implement basic text processing using natural language processing.	8-08-2026	
9	To demonstrate a fundamental image processing technique by splitting a multi-channel color image into its individual color components and visualizing the contribution of each channel to the overall image.	22-08-2026	
10	To determine the edges of an image.	22-08-2026	
11	To apply a thumbnail function to an image.	22-08-2026	
12	To predict the price of any item using supervised learning algorithm.	5-09-2026	
13	To predict the label using an unsupervised learning algorithm.	5-09-2026	
14	To find the correlation between the two datasets.	12-09-2026	

PRACTICAL 1 :

A] AIM: To extract data from an image and transform it into a structured, machine-readable CSV format.

CONCEPT :

Image-to-data extraction is the process of converting the information stored in an image into a structured and machine-readable format. Each pixel of an image contains color information such as **Red, Green, Blue (RGB)** and sometimes **Alpha (transparency)** values. By extracting the pixel values along with their **X and Y coordinates**, the image can be represented as a structured dataset. This data can then be stored in **CSV format**, making it easier to analyze, process, and use in data science applications.

CODE :

Step 1: Upload Image

```
from google.colab import files

uploaded = files.upload()
```

Step 2: Import Libraries

```
from skimage import io

import pandas as pd

import matplotlib.pyplot as plt

import numpy as np
```

Step 3: Read Image

```
image_name = list(uploaded.keys())[0]

image = io.imread(image_name)
```

Step 4: Display Image

```
plt.imshow(image)

plt.title("Input Image")
```

```
plt.axis("off")
plt.show()
```

Step 5: Display Image Information

```
print("Image Shape :", image.shape)
print("Height :", image.shape[0])
print("Width :", image.shape[1])
if image.shape[2] == 3:
print("Color Mode : RGB")
else:
print("Color Mode : RGBA")
```

Step 6: Extract Pixel Data

```
height, width, channels = image.shape
rows = []
for y in range(height):
for x in range(width):
pixel = image[y, x]
if channels == 3:
r, g, b = pixel
a = 255
else:
r, g, b, a = pixel
rows.append([x, y, r, g, b, a])
```

Step 7: Create DataFrame

```
df = pd.DataFrame(
rows,
```

```
columns=["X", "Y", "Red", "Green", "Blue", "Alpha"]
)
```

Step 8: Display First 10 Records

```
print(df.head(10))
```

Step 9: Display Dataset Information

```
print("\nRows :", df.shape[0])

print("Columns :", df.shape[1])
```

Step 10: Save as CSV

```
output_file = "Image_to_HORUS.csv"
df.to_csv(output_file, index=False)
print("\nCSV File Saved Successfully.")
```

Step 11: Download CSV

```
files.download(output_file)
```

OUTPUT :

Input Image



```
Image Shape : (183, 275, 3)
Height : 183
Width : 275
Color Mode : RGB
   X  Y  Red  Green  Blue  Alpha
0  0  0   47    103   162    255
1  1  0   48    104   163    255
2  2  0   44    100   159    255
3  3  0   36     92   151    255
4  4  0   37     93   152    255
5  5  0   42     98   157    255
6  6  0   36     92   151    255
7  7  0   24     80   139    255
8  8  0   24     64   116    255
9  9  0   23     65   113    255

Rows : 50325
Columns : 6

CSV File Saved Successfully.
```

B] AIM : To process and prepare raw CSV data to conform to the "HORUS" standard, making it suitable for analysis.

CONCEPT :

Data preprocessing is the process of cleaning and organizing raw CSV data so that it can be used effectively for analysis. In the **HORUS standard**, the data is arranged in a consistent and structured format with proper column names, data types, and values. This process may include removing missing or invalid data, correcting inconsistencies, and organizing the dataset. Proper preprocessing improves **data quality, accuracy, and reliability**, making the data suitable for further analysis and processing.

CODE :

Python Code

```
import pandas as pd
# Input File
sInputFileName = 'Country.csv'
InputData = pd.read_csv(
    sInputFileName,
    encoding="latin-1"
)
print("Input Data Values")
print(InputData)
print("=" * 60)
# ============================================
# Processing Rules
# ============================================
ProcessData = InputData.copy()
# Remove unwanted columns
ProcessData.drop(
    columns=['ISO-2-CODE', 'ISO-3-Code'],
    inplace=True
```

```
)
# Rename columns
ProcessData.rename(
    columns={
        'Country': 'CountryName',
        'ISO-M49': 'CountryNumber'
    },
    inplace=True
)
# Set CountryNumber as Index
ProcessData.set_index(
    'CountryNumber',
    inplace=True
)
# Sort CountryName in Ascending Order
ProcessData.sort_values(
    by='CountryName',
    ascending=True,
    inplace=True
)
print("Processed Data")
print(ProcessData)
print("=" * 60)

# ========================================
# Output Agreement
# ========================================
sOutputFileName = 'HORUS-CSV-Country.csv'
ProcessData.to_csv(sOutputFileName)
```

```
print("CSV to HORUS - Done Successfully")
```

OUTPUT :

```
Input Data Values
                        Country ISO-2-CODE ISO-3-Code  ISO-M49
0                   Afghanistan         AF        AFG        4
1                 Aland Islands         AX        ALA      248
2                       Albania         AL        ALB        8
3                       Algeria         DZ        DZA       12
4                  American Samoa         AS        ASM       16
..                            ...        ...        ...      ...
242  Wallis and Futuna Islands         WF        WLF      876
243              Western Sahara         EH        ESH      732
244                       Yemen         YE        YEM      887
245                      Zambia         ZM        ZMB      894
246                    Zimbabwe         ZW        ZWE      716

[247 rows x 4 columns]
============================================================
Processed Data
                              CountryName
CountryNumber
4                               Afghanistan
248                           Aland Islands
8                                   Albania
12                                  Algeria
16                           American Samoa
...                                     ...
876               Wallis and Futuna Islands
732                          Western Sahara
887                                   Yemen
894                                  Zambia
716                                Zimbabwe

[247 rows x 1 columns]
============================================================
CSV to HORUS - Done Successfully
```

PRACTICAL 2

A)AIM : Removing Leading, Lagging or extra spaces from a data entry.

CODE :

```
cleandata = ' '.join(baddata.split())
```

baddata.split() → breaks the text into words by removing all extra spaces.

B)AIM : Removing non-printable characters .

CODE :

string.printable is a string of all printable characters.

- `\x00` → Null character
- `\x02` → Start of text
- `\x10` → Data link escape
- These are **control characters** that you usually don't want in your data.

```
cleandata = ''.join(filter(lambda x: x in string.printable, baddata))
```

`filter(...)` → goes through each character in `baddata`.

`lambda x: x in string.printable` → keeps only the characters that are in the **printable set**.

`''.join(...)` → joins the filtered characters into a **clean string**.

C)AIM : Reformatting Data Entry.

CODE :

• import datetime

•Python's datetime module for working with date and time.

```
baddate = datetime.date(2019, 10, 31)
```

Creates a **date object** for **October 31, 2019**.

```
baddata = format(baddate, '%Y-%m-%d')
```

Converts the date into a **string** in the format `YYYY-MM-DD`.

- So `baddata` → `'2019-10-31'`

Here it's called **bad data** only because the format is not what you want.

```
gooddate = datetime.datetime.strptime(baddata, '%Y-%m-%d')
```

`strptime` converts the **string** `'2019-10-31'` back into a **datetime object**, using the given format.

D)AIM : To detect outliers using a fixed latitude range.

CODE :

Loads the CSV file into a **Pandas DataFrame**.

It reads only **4 columns**: `'Country'`, `'Place.Name'`, `'Latitude'`, `'Longitude'`.

`encoding="latin-1"` is used to handle **special characters**.

`low_memory=False` prevents warnings for mixed data types.

```
LondonData = IP_DATA_ALL.loc[IP_DATA_ALL['Place_Name'] == 'London']
```

Filters the data to include **only rows** where the place is **London**.

E)AIM : To calculate average latitude by country and place name.

CODE :

Creates a smaller DataFrame with only:

•'Country'

•'Place_Name'

•'Latitude'

Changes 'Place.Name' to 'Place_Name' to make the column name easier to work with (no dot).

F)AIM : To visualize data binning with a histogram and normal distribution fit.

CODE :

•Instead of working with every possible value, you divide the range into bins. •Each bin represents a range of values, and the original values are replaced by the bin label.

EXAMPLE—BINNING

```
Ages: 13, 15, 18, 21, 24, 29, 35, 40, 47, 53, 60, 68, 7
```

Binning:

```yaml
Bin 1: 10–20 → Teenager
Bin 2: 21–40 → Young Adult
Bin 3: 41–60 → Middle-aged Adult
Bin 4: 61–80 → Senior
```

PRACTICAL 3 :

AIM : To understand the use of Retrieve Superstep.

CONCEPT :

The **Retrieve Superstep** is used to fetch or retrieve required data from a stored dataset or database for further processing. It helps the system access specific records or information based on given conditions or requirements. The retrieved data can then be filtered, analyzed, or passed to the next processing step. This makes data processing **faster, organized, and efficient**, especially when working with large datasets.

CODE :

```
# To extract Unique Router Locations from IP Data and Save to CSV”
#######################################################################
# -*- coding: utf-8 -*-
#######################################################################
import os
import pandas as pd
#######################################################################
InputFileName='IP_DATA_ALL.csv'
OutputFileName='Retrieve_Router_Location.csv'
#######################################################################
#######################################################################
sFileName="/content/IP_DATA_ALL.csv"
print('Loading :',sFileName)
IP_DATA_ALL=pd.read_csv(sFileName,header=0,low_memory=False,
 usecols=['Country','Place.Name','Latitude','Longitude'])
IP_DATA_ALL.rename(columns={'Place.Name': 'Place_Name'}, inplace=True)
#######################################################################
sFileDir='/content/'
ROUTERLOC = IP_DATA_ALL.drop_duplicates(subset=None, keep='first',
inplace=False)
```

```
print('Rows :',ROUTERLOC.shape[0])
print('Columns :',ROUTERLOC.shape[1])
sFileName2=sFileDir + '/' + OutputFileName
ROUTERLOC.to_csv(sFileName2, index = False)
#########################################################################
print('### Done!! ################################################')
#########################################################################
```

OUTPUT :

```
Loading : /content/IP_DATA_ALL.csv
Rows : 37452
Columns : 4
### Done!! ############################################
```

PRACTICAL 4 :

AIM : To understand the use of histogram,pie chart and line chart in data science.

CONCEPT :

Charts are important tools in **Data Science** for presenting and understanding data visually. A **histogram** is used to show the distribution and frequency of numerical data, helping to identify patterns and ranges. A **pie chart** represents data as parts of a whole and is useful for comparing percentages or proportions. A **line chart** shows changes and trends in data over time or across an ordered sequence. These charts make complex data **easier to understand, compare, and interpret.**

CODE :

HISTOGRAM USING MATPLOTLIB

```
# Importing matplotlib module

from matplotlib import pyplot as plt

# Y-axis values

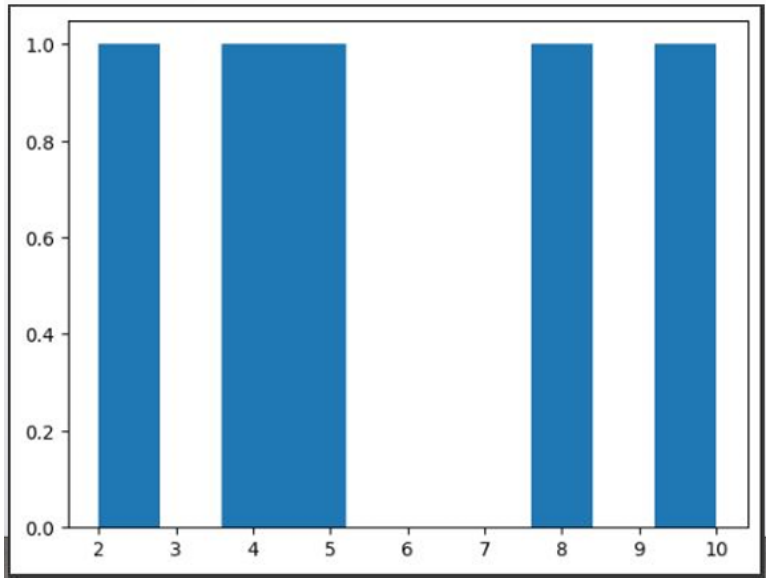
y = [10, 5, 8, 4, 2]

# Function to plot histogram

plt.hist(y)

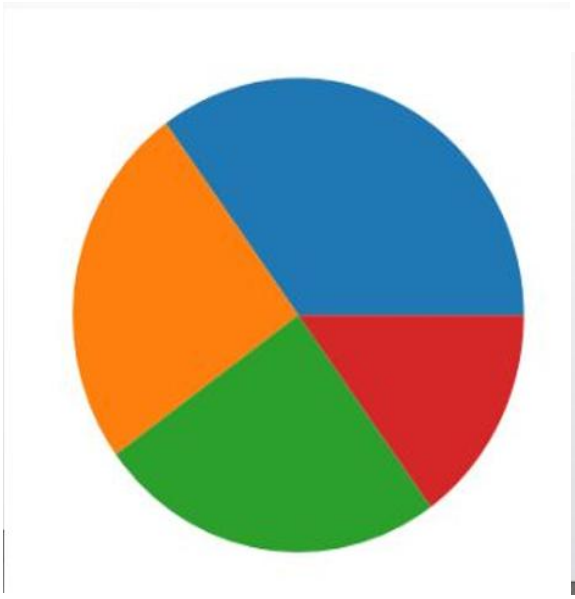
# Function to show the plot

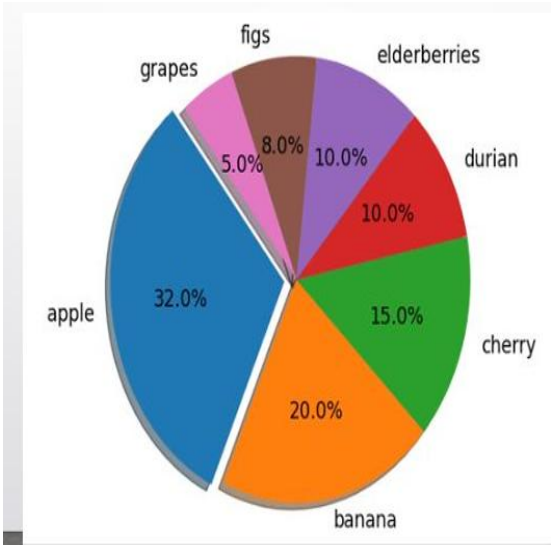
plt.show()
```



PIE CHART USING MATPLOTLIB

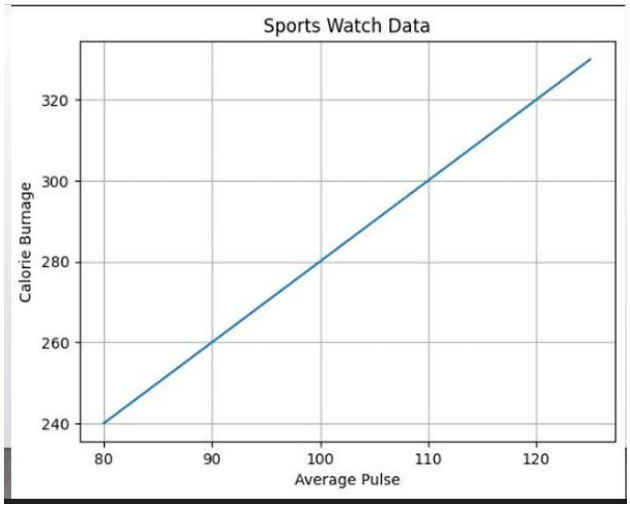
```
# Importing required libraries
import matplotlib.pyplot as plt
import numpy as np
y = np.array([35, 25, 25, 15])
plt.pie(y)
# Display the pie chart
plt.show()
import matplotlib.pyplot as plt
labels = ['apple', 'banana', 'cherry', 'durian', 'elderberries', 'figs', 'grapes']
sizes = [32, 20, 15, 10, 10, 8, 5]
p = plt.pie(sizes, labels=labels, explode=(0.07, 0, 0, 0, 0, 0, 0),
        autopct='%1.1f%%', startangle=130, shadow=True)
plt.axis('equal')
for i, (apple, banana, cherry, durian, elderberries, figs, grapes) in enumerate(p):
   if i > 0:
      apple.set_fontsize(12)
      banana.set_fontsize(12)
      cherry.set_fontsize(12)
      durian.set_fontsize(12)
      elderberries.set_fontsize(12)
      figs.set_fontsize(12)
      grapes.set_fontsize(12)
plt.show()
```





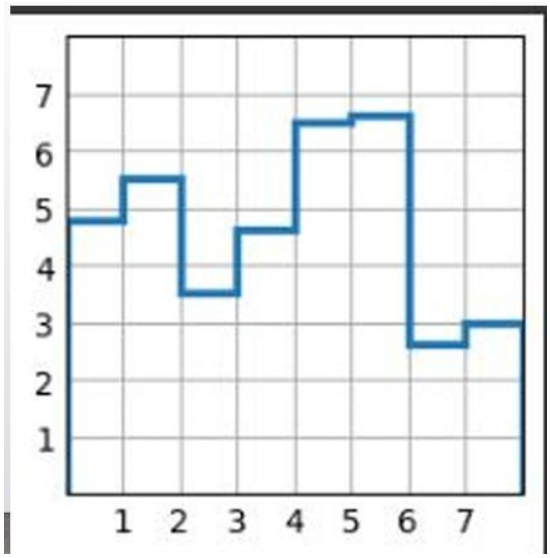
ADDING GRID LINES TO PLOT

```
# Importing required libraries
import numpy as np
import matplotlib.pyplot as plt
# X-axis values
x = np.array([80, 85, 90, 95, 100, 105, 110, 115, 120, 125])
# Y-axis values
y = np.array([240, 250, 260, 270, 280, 290, 300, 310, 320, 330])
# Add title to the plot
plt.title("Sports Watch Data")
# Add labels to the axes
plt.xlabel("Average Pulse")
plt.ylabel("Calorie Burnage")
# Plot the data
plt.plot(x, y)
# Add grid lines
plt.grid()
plt.show()
```



STAIRS(VALUES)

```
import matplotlib.pyplot as plt
import numpy as np
plt.style.use('_mpl-gallery')
# make data
y = [4.8, 5.5, 3.5, 4.6, 6.5, 6.6, 2.6, 3.0]
# plot
fig, ax = plt.subplots()
ax.stairs(y, linewidth=2.5)
ax.set(xlim=(0, 8), xticks=np.arange(1, 8),
       ylim=(0, 8), yticks=np.arange(1, 8)) plt.show()
```



PRACTICAL 5

AIM : To implement complex data visualizations.

CONCEPT :

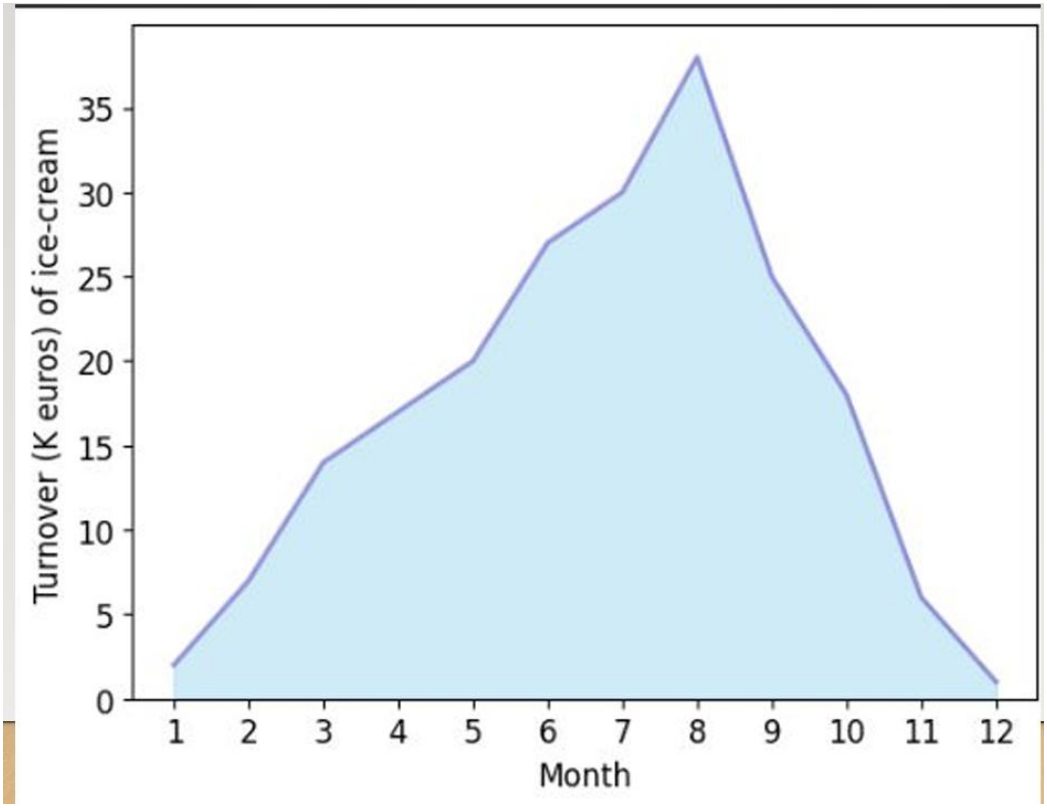
Complex data visualization is the process of representing large or multidimensional datasets using graphical techniques. It helps users understand **relationships, patterns, trends, and variations** that may be difficult to identify from raw data. Tools such as **Matplotlib, Seaborn, and Plotly** can be used to create advanced charts and graphs. Complex visualizations make data analysis more **clear, interactive, and meaningful**, helping in better interpretation and decision-making.

CODE :

AREA PLOT USING MATPLOTLIB

```
# Importing required libraries
import datetime
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
# Turnover data
turnover = [2, 7, 14, 17, 20, 27, 30, 38, 25, 18, 6, 1]
# Fill the area under the plot
plt.fill_between(
    np.arange(12),
    turnover,
    color="skyblue",
    alpha=0.4
)
# Plot the turnover data
plt.plot(
    np.arange(12),
    turnover,
```

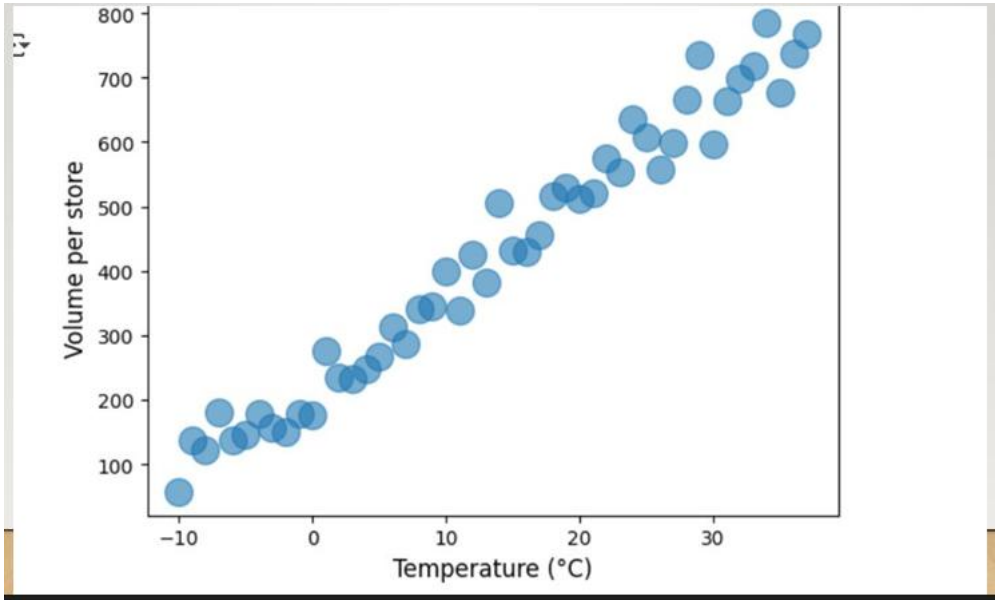
```
color="Slateblue",
    alpha=0.6,
    linewidth=2
)
# Set tick label size
plt.tick_params(labelsize=12)
# Set X-axis values
plt.xticks(
    np.arange(12),
    np.arange(1, 13)
)
# Add X-axis label
plt.xlabel(
    'Month',
    size=12
)
# Add Y-axis label
plt.ylabel(
    'Turnover (K euros) of ice-cream',
    size=12
)
# Set the lower limit of Y-axis
plt.ylim(bottom=0)
# Display the plot
plt.show()
```



SCATTER PLOT USING MATPLOTLIB

```
# Importing required libraries
import numpy as np
import matplotlib.pyplot as plt
# X-axis: Temperatures from -10°C to 37°C
x = range(-10, 38, 1)
# Y-axis: Sales volume per store
# Starts low, increases, plus random noise
y = np.array(range(60, 770, 15)) + np.random.randn(48) * 40
# Create scatter plot
plt.scatter(
    x,
    y,
    s=200,       # Dot size
    alpha=0.6    # Dot transparency
)
```

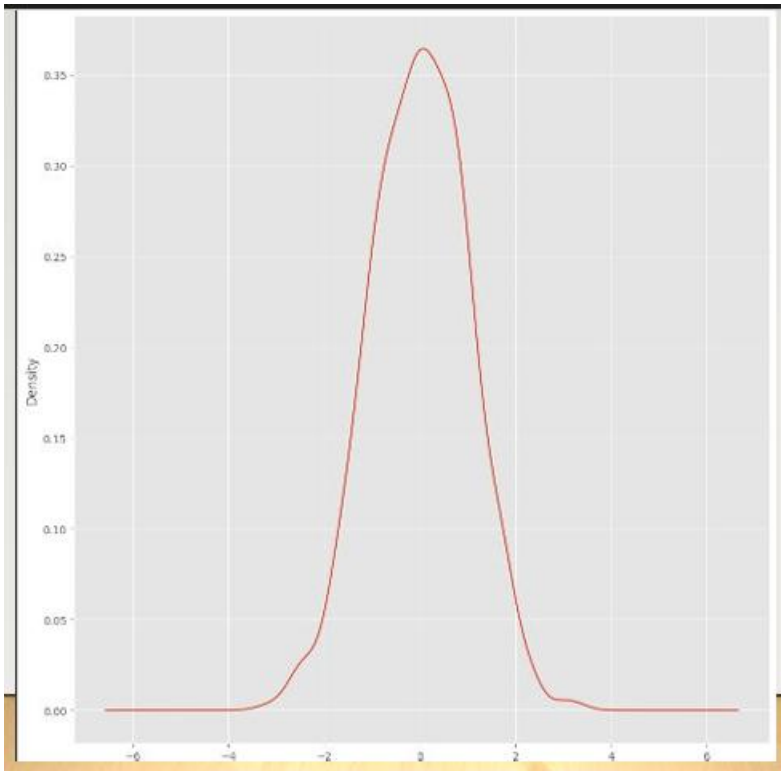
```
plt.xlabel(
    'Temperature (°C)',
    size=12
)
# Add Y-axis label
plt.ylabel(
    'Volume per store',
    size=12
)
# Display the plot
plt.show()
```



KDE

```
import sys
import os
import pandas as pd
import matplotlib as ml
import numpy as np
from matplotlib import pyplot as plt
ml.style.use('ggplot')
```

```
fig1=plt.figure(figsize=(10, 10))
ser = pd.Series(np.random.randn(1000))
ser.plot(figsize=(10, 10), kind='kde')
sPicNameOut1='/content/kde.png'
plt.savefig(sPicNameOut1, dpi=600)
plt.tight_layout()
plt.show()
```



SCATTERPLOT MATRIX

```
import sys
import os
import pandas as pd
import matplotlib as ml
import numpy as np
from matplotlib import pyplot as plt
fig2=plt.figure(figsize=(10, 10))
from pandas.plotting import scatter_matrix
```

```
df = pd.DataFrame(np.random.randn(1000, 5), columns=['Y2014', 'Y2015','Y2016', 'Y2017', 'Y2018'])

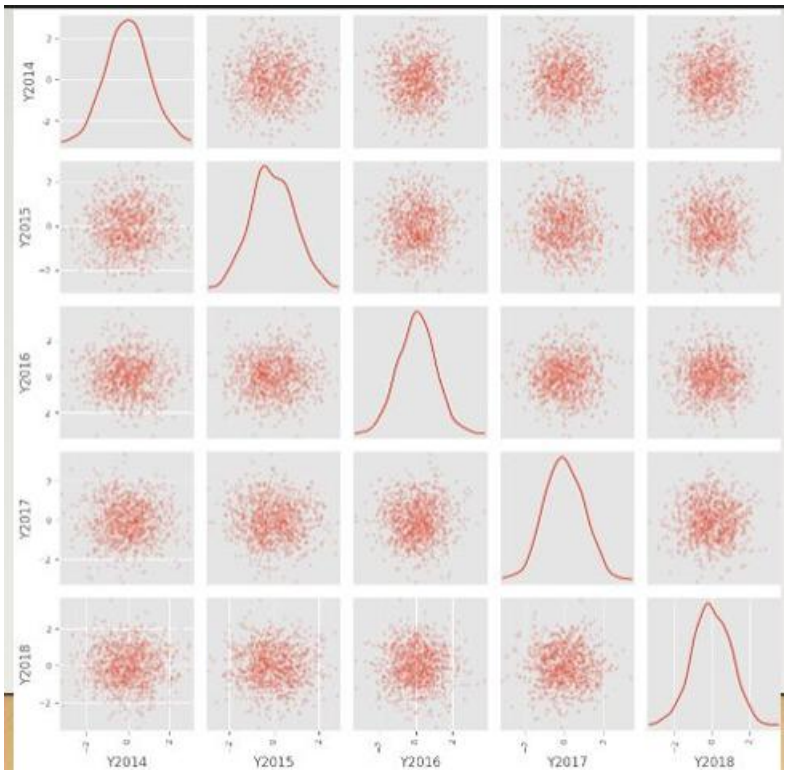
scatter_matrix(df, alpha=0.2, figsize=(10, 10), diagonal='kde')

sPicNameOut2='/content/scatter_matrix.png'

plt.savefig(sPicNameOut2,dpi=600)

plt.tight_layout()

plt.show()
```



PARALLEL COORDINATES

```
import sys

import os

import pandas as pd

import matplotlib as ml

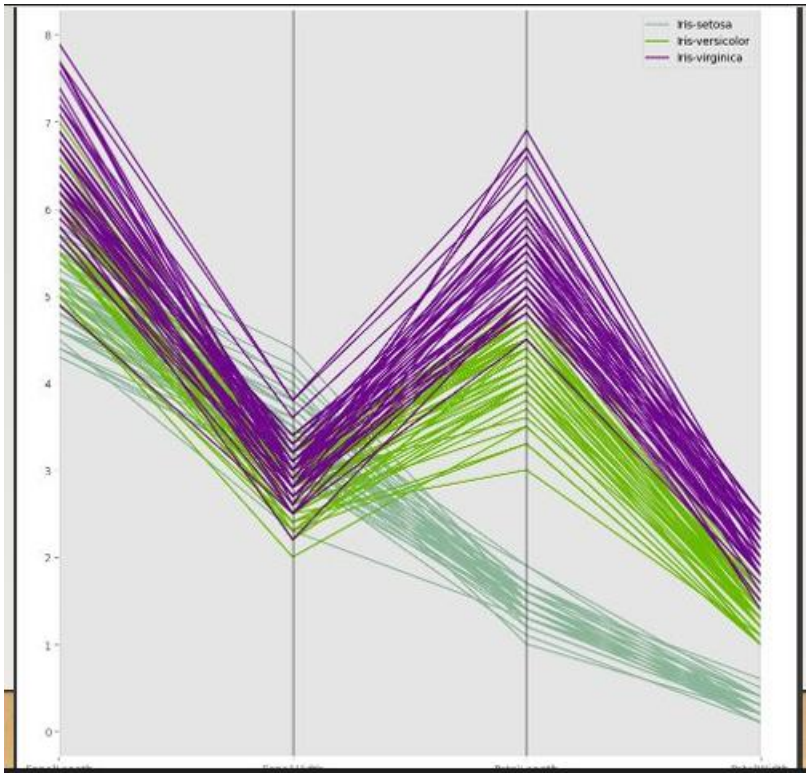
import numpy as np

from matplotlib import pyplot as plt

from pandas.plotting import parallel_coordinates

plt.figure(figsize=(10, 10))
```

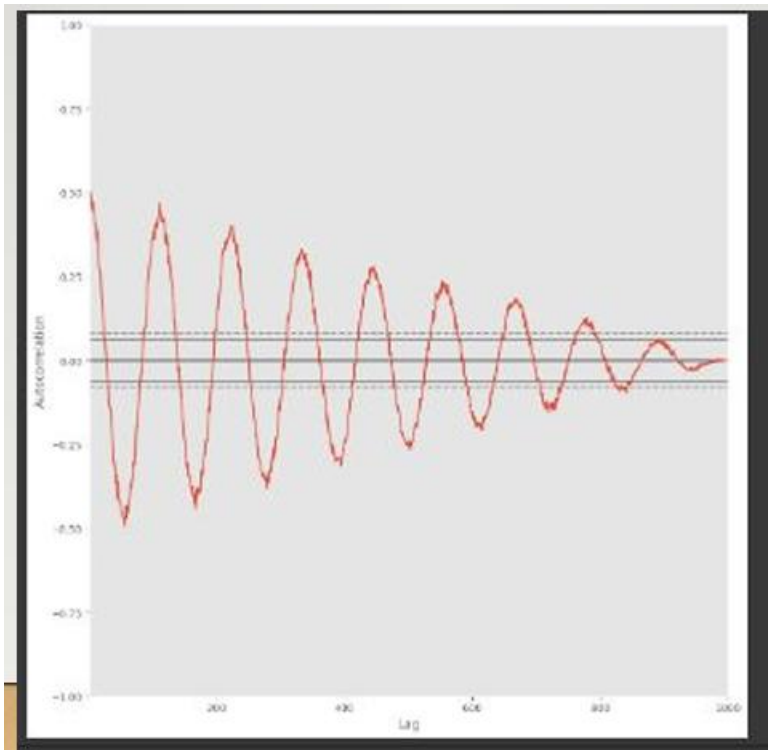
```
sDataFile='/content/iris1.csv'
data = pd.read_csv(sDataFile)
parallel_coordinates(data, 'Name')
sPicNameOut2='/content/parallel_coordinates.png'
plt.savefig(sPicNameOut2,dpi=600)
plt.tight_layout()
plt.show()
```



Autocorrelation Plot :

```
import sys
import os
import pandas as pd
import matplotlib as ml
import numpy as np
from matplotlib import pyplot as plt
from pandas.plotting import autocorrelation_plot
```

```
plt.figure(figsize=(10, 10))
data = pd.Series(0.7 * np.random.rand(1000) + \
0.3 * np.sin(np.linspace(-9 * np.pi, 9 * np.pi, num=1000)))
autocorrelation_plot(data)
sPicNameOut2='/content/autocorrelation_plot.png'
plt.savefig(sPicNameOut2,dpi=600)
plt.tight_layout()
plt.show()
```



PRACTICAL 6

AIM : To understand the use of Assess superstep.

CONCEPT :

The **Assess superstep** is used to evaluate the performance of a machine learning model. It compares the predicted results with the actual results using suitable evaluation metrics. It helps determine how accurately the model performs and identifies areas for improvement.

CODE :

```
# 1. REPLACING NaN VALUES WITH MEAN
import pandas as pd
import numpy as np
df = pd.DataFrame(
    [
        [10, np.nan, 30, 40],
        [7, 14, 21, 28],
        [55, np.nan, 8, 12],
        [15, 14, np.nan, 8],
        [7, 1, 1, np.nan],
        [np.nan, 4, 9, 2]
    ],
    columns=['Apple', 'Orange', 'Banana', 'Pear'],
    index=[
        'Basket1',
        'Basket2',
        'Basket3',
        'Basket4',
        'Basket5',
        'Basket6'
    ]
)
```

```
print("###################################")
print("THE ORIGINAL VALUES")
print(df)
print("REPLACING THE VALUES WITH MEAN")
df.fillna(
   df.mean(),
   inplace=True
)
print(df)
```

```
###################################
THE ORIGINAL VALUES
         Apple  Orange  Banana  Pear
Basket1   10.0     NaN    30.0  40.0
Basket2    7.0    14.0    21.0  28.0
Basket3   55.0     NaN     8.0  12.0
Basket4   15.0    14.0     NaN   8.0
Basket5    7.0     1.0     1.0   NaN
Basket6    NaN     4.0     9.0   2.0
REPLACING THE VALUES WITH MEAN
         Apple  Orange  Banana  Pear
Basket1   10.0    8.25    30.0  40.0
Basket2    7.0   14.00    21.0  28.0
Basket3   55.0    8.25     8.0  12.0
Basket4   15.0   14.00    13.8   8.0
Basket5    7.0    1.00     1.0  18.0
Basket6   18.8    4.00     9.0   2.0
```

#2.REPLACING NAN VALUES WITH MEDIAN

```
import pandas as pd
import numpy as np
df = pd.DataFrame(
   [
      [10, np.nan, 30, 40],
      [7, 14, 21, 28],
      [55, np.nan, 8, 12],
      [15, 14, np.nan, 8],
      [7, 1, 1, np.nan],
```

```
        [np.nan, 4, 9, 2]
    ],
    columns=['Apple', 'Orange', 'Banana', 'Pear'],
    index=[
        'Basket1',
        'Basket2',
        'Basket3',
        'Basket4',
        'Basket5',
        'Basket6'
    ]
)
print("###################################")
print("THE ORIGINAL VALUES")
print(df)
print("REPLACING THE VALUES WITH MEDIAN")
df.fillna(
    df.median(),
    inplace=True
)
print(df)
```

```
###################################
THE ORIGINAL VALUES
         Apple  Orange  Banana  Pear
Basket1   10.0     NaN    30.0  40.0
Basket2    7.0    14.0    21.0  28.0
Basket3   55.0     NaN     8.0  12.0
Basket4   15.0    14.0     NaN   8.0
Basket5    7.0     1.0     1.0   NaN
Basket6    NaN     4.0     9.0   2.0
REPLACING THE VALUES WITH MEDIAN
         Apple  Orange  Banana  Pear
Basket1   10.0     9.0    30.0  40.0
Basket2    7.0    14.0    21.0  28.0
Basket3   55.0     9.0     8.0  12.0
Basket4   15.0    14.0     9.0   8.0
Basket5    7.0     1.0     1.0  12.0
Basket6   10.0     4.0     9.0   2.0
```

#3.REPLACING NAN VALUES WITH MODE

```
import pandas as pd
import numpy as np
df = pd.DataFrame(
    [
        [10, np.nan, 30, 40],
        [7, 14, 21, 28],
        [55, np.nan, 8, 12],
        [15, 14, np.nan, 8],
        [7, 1, 1, np.nan],
        [np.nan, 4, 9, 2]
    ],
    columns=['Apple', 'Orange', 'Banana', 'Pear'],
    index=[
        'Basket1',
        'Basket2',
        'Basket3',
        'Basket4',
        'Basket5',
        'Basket6'
    ]
)
print("####################################")
print("THE ORIGINAL VALUES")
print(df)
print("REPLACING THE VALUES WITH MODE")
for column in df.columns:
    df[column] = df[column].fillna(
        df[column].mode()[0]
```

```
    )
print(df)
```

```
##################################
THE ORIGINAL VALUES
         Apple  Orange  Banana  Pear
Basket1   10.0     NaN    30.0  40.0
Basket2    7.0    14.0    21.0  28.0
Basket3   55.0     NaN     8.0  12.0
Basket4   15.0    14.0     NaN   8.0
Basket5    7.0     1.0     1.0   NaN
Basket6    NaN     4.0     9.0   2.0
REPLACING THE VALUES WITH MODE
         Apple  Orange  Banana  Pear
Basket1   10.0    14.0    30.0  40.0
Basket2    7.0    14.0    21.0  28.0
Basket3   55.0    14.0     8.0  12.0
Basket4   15.0    14.0     1.0   8.0
Basket5    7.0     1.0     1.0   2.0
Basket6    7.0     4.0     9.0   2.0
```

#4.REPLACING NAN VALUES WITH MINIMUM

```
import pandas as pd
import numpy as np
df = pd.DataFrame(
    [
        [10, np.nan, 30, 40],
        [7, 14, 21, 28],
        [55, np.nan, 8, 12],
        [15, 14, np.nan, 8],
        [7, 1, 1, np.nan],
        [np.nan, 4, 9, 2]
    ],
    columns=['Apple', 'Orange', 'Banana', 'Pear'],
    index=[
        'Basket1',
        'Basket2',
```

```
        'Basket3',
        'Basket4',
        'Basket5',
        'Basket6'
    ]
)
print("###################################")
print("THE ORIGINAL VALUES")
print(df)
print("REPLACING THE VALUES WITH MINIMUM")
df.fillna(
    df.min(),
    inplace=True
)
print(df)
```

```
###################################
THE ORIGINAL VALUES
         Apple  Orange  Banana  Pear
Basket1   10.0     NaN    30.0  40.0
Basket2    7.0    14.0    21.0  28.0
Basket3   55.0     NaN     8.0  12.0
Basket4   15.0    14.0     NaN   8.0
Basket5    7.0     1.0     1.0   NaN
Basket6    NaN     4.0     9.0   2.0
REPLACING THE VALUES WITH MINIMUM
         Apple  Orange  Banana  Pear
Basket1   10.0     1.0    30.0  40.0
Basket2    7.0    14.0    21.0  28.0
Basket3   55.0     1.0     8.0  12.0
Basket4   15.0    14.0     1.0   8.0
Basket5    7.0     1.0     1.0   2.0
Basket6    7.0     4.0     9.0   2.0
```

PRACTICAL 7

AIM : To implement line detection using hough transform.

CONCEPT :

Hough Transform is a computer vision technique to detect shapes like lines and circles in an image. It converts these shapes into mathematical representations in a parameter space, making it easier to identify them even if they're broken or obscured. This method is valuable for image analysis, pattern recognition, and object detection.

The HT is a feature extraction method in image analysis, computer vision, and digital image processing .It uses a voting mechanism to identify bad examples of objects inside a given class of forms. This voting mechanism is carried out in parameter space. First, object candidates are produced as local maxima in an accumulator space, using the HT algorithm.

CODE :

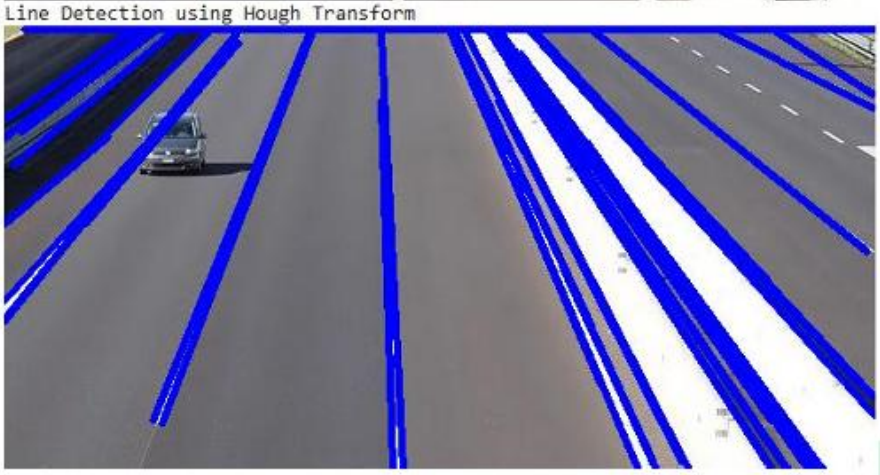
```
# Import required libraries
import cv2
import numpy as np
from google.colab.patches
 import cv2_imshow
# Read the image
img = cv2.imread("hough.jpg")
# Display the original image
cv2_imshow(img)
# Convert image to grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
# Detect edges using Canny Edge Detection
edges = cv2.Canny(gray, 50, 200)
# Detect lines using Probabilistic Hough Transform
lines = cv2.HoughLinesP(
    edges,
    1,
    np.pi / 180,
```

```
    68,
    minLineLength=15,
    maxLineGap=250
)
# Check if lines are detected
if lines is not None:
    for line in lines:
        x1, y1, x2, y2 = line.flatten()
        # Draw detected lines on the image
        cv2.line(
            img,
            (x1, y1),
            (x2, y2),
            (255, 0, 0),
            3
        )
else:
    print("No lines detected.")
# Display the result
print("Line Detection using Hough Transform")
cv2_imshow(img)
```

OUTPUT :



Line Detection using Hough Transform



PRACTICAL 8

AIM : To implement basic text processing using natural language processing.

CONCEPT :

Natural Language Processing (NLP) is a field of Artificial Intelligence that enables machines to interpret, and generate human language. It bridges the gap between human communication and computer understanding using techniques like text analysis, tokenization, and language modeling.This is a widely used technology for personal assistants that are used in various business fields/areas. NLP is used in a wide range of applications, including machine translation, sentiment analysis, speech recognition, chatbots, and text classification.

CODE :

```
# Import NLTK

import nltk

# Download required NLTK resources safely

nltk.download('punkt')

nltk.download('stopwords')

nltk.download('punkt_tab')

# Import necessary libraries

from nltk.tokenize import sent_tokenize, word_tokenize

from nltk.corpus import stopwords

from nltk.probability import FreqDist

# Sample text

text = """

Natural Language Processing (NLP) is a field of Artificial Intelligence.

It focuses on the interaction between computers and human language.

NLP helps machines understand, interpret, and generate human language.

"""

# Step 1: Sentence Tokenization

print("Sentence Tokenization:")
```

```
sentences = sent_tokenize(text)
print(sentences, '\n')
# Step 2: Word Tokenization
print("Word Tokenization:")
words = word_tokenize(text)
print(words, '\n')
# Step 3: Stopword Removal
print("Removing Stopwords:")
stop_words = set(stopwords.words('english'))
filtered_words = [
    word
    for word in words
    if word.lower() not in stop_words and word.isalnum()
]
print(filtered_words, '\n')
# Step 4: Frequency Distribution
print("Frequency Distribution:")
fdist = FreqDist(filtered_words)
for word, freq in fdist.most_common():
    print(f"{word}: {freq}")
```

Output :

Sentence Tokenization

['Natural Language Processing (NLP) is a field of Artificial Intelligence.',
'It focuses on the interaction between computers and human language.',
'NLP helps machines understand, interpret, and generate human language.']

Word Tokenization

['Natural', 'Language', 'Processing', '(', 'NLP', ')', 'is', 'a',
'field', 'of', 'Artificial', 'Intelligence', '.', 'It', 'focuses',
'on', 'the', 'interaction', 'between', 'computers', 'and', 'human',
'language', '.', 'NLP', 'helps', 'machines', 'understand', ',',
'interpret', ',', 'and', 'generate', 'human', 'language', '.']

Removing Stopwords

['Natural', 'Language', 'Processing', 'NLP', 'field', 'Artificial',
'Intelligence', 'focuses', 'interaction', 'computers', 'human',
'language', 'NLP', 'helps', 'machines', 'understand', 'interpret',
'generate', 'human', 'language']

Frequency Distribution

NLP: 2

human: 2

language: 2

Natural: 1

Language: 1

Processing: 1

field: 1

Artificial: 1

Intelligence: 1

focuses: 1

interaction: 1

computers: 1

helps: 1

machines: 1

understand: 1

interpret: 1

generate: 1

NLTK Download Output

[nltk_data] Downloading package punkt to /root/nltk_data...

[nltk_data] Package punkt is already up-to-date!

[nltk_data] Downloading package stopwords to /root/nltk_data...

[nltk_data] Package stopwords is already up-to-date!

[nltk_data] Downloading package punkt_tab to /root/nltk_data...

[nltk_data] Unzipping tokenizers/punkt_tab.zip.

PRACTICAL 9

AIM : To demonstrate a fundamental image processing technique by splitting a multi-channel color image into its individual color components and visualizing the contribution of each channel to the overall image.

CONCEPT :

Color Channel Splitting is a basic image processing technique used to separate a color image into its individual **Red, Green, and Blue (RGB) channels**. Each channel represents the intensity of one primary color in the image. The separated channels are usually displayed as grayscale images, where bright areas indicate higher intensity. This technique helps in analyzing the contribution of each color channel to the original image.

CODE :

```
import sys

import os

import matplotlib.pyplot as plt

import matplotlib.image as mpimg

sPicName='/content/1.png'

t=0

img=mpimg.imread(sPicName)

print('Size:', img.shape)

plt.figure(figsize=(10, 10))

t+=1

sTitle='(' + str(t) + ') Original'

plt.title(sTitle)

plt.imshow(img)

plt.show()

for c in range(img.shape[2]):

    t+=1

    plt.figure(figsize=(10, 10))

    sTitle='(' + str(t) + ') Channel: ' + str(c)

    plt.title(sTitle)
```

```
lum_img = img[:,:,c]
plt.imshow(lum_img)
plt.show()
```

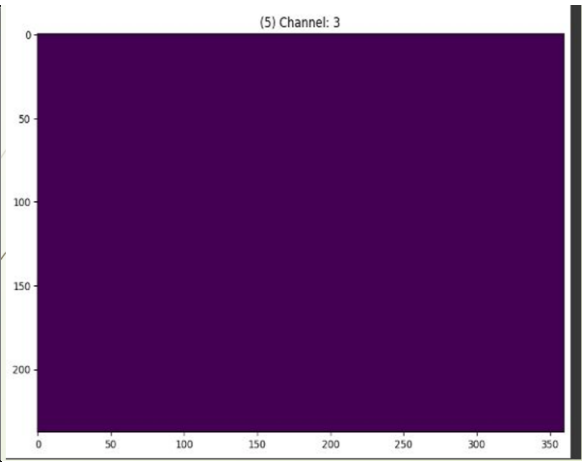
OUTPUT :











PRACTICAL 10

AIM : To determine the edges of an image.

CONCEPT :

Edge detection is an image processing technique used to identify the **boundaries and sharp changes in intensity** within an image. It helps locate important features such as object outlines and shapes. Common edge detection methods include **Sobel, Prewitt, and Canny** operators. It is widely used in **object detection, image segmentation, and shape recognition**.

CODE :

```
import sys

import os

import matplotlib.pyplot as plt

from PIL import Image

sPicNameIn='/content/1.png'

sPicNameOut='/content/audi.png'

imageIn = Image.open(sPicNameIn)

fig1=plt.figure(figsize=(10, 10))

fig1.suptitle('Audi R8', fontsize=20)

imgplot = plt.imshow(imageIn)

mask=imageIn.convert("L")

th=49

imageOut = mask.point(lambda i: i < th and 255)

imageOut.save(sPicNameOut)

imageTest = Image.open(sPicNameOut)

fig2=plt.figure(figsize=(10, 10))

fig2.suptitle('Audi R8 Edge', fontsize=20)

imgplot = plt.imshow(imageTest)
```

OUTPUT :





PRACTICAL 11

AIM : To apply a thumbnail function to an image.

CONCEPT :

A thumbnail is a small-sized version of an original image. The thumbnail function reduces the image dimensions while maintaining its overall appearance. It is useful for displaying images quickly and saving storage space. Image thumbnails are commonly used in galleries, websites, and image browsers.

CODE :

```
import sys

import os

import matplotlib.pyplot as plt

from PIL import Image

sPicName='/content/1.png'

nSize=4

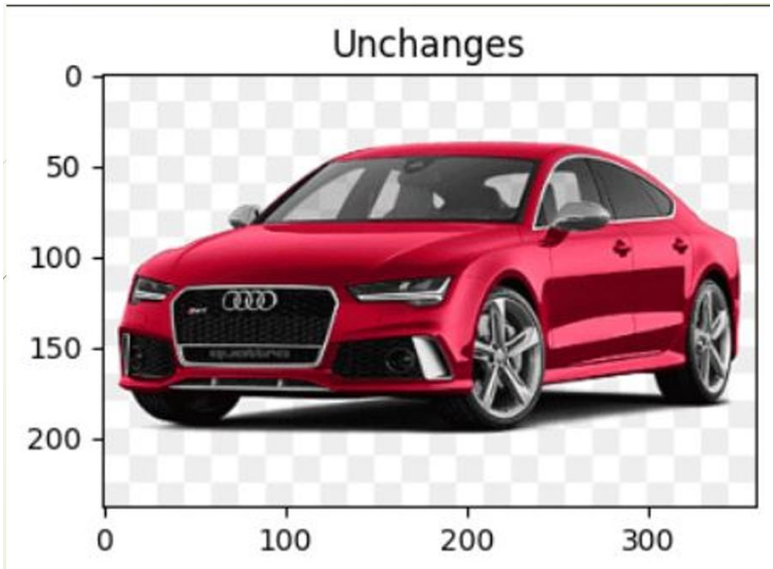
img = Image.open(sPicName)

plt.figure(figsize=(nSize, nSize))

sTitle='Unchances'

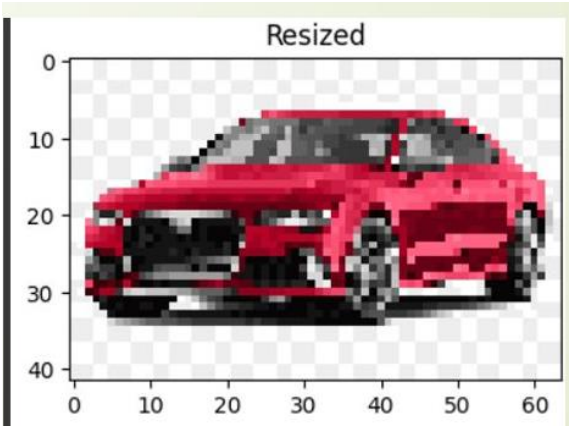
plt.title(sTitle)

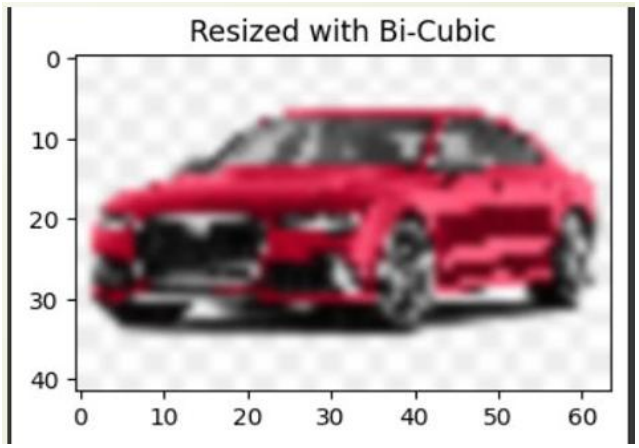
imgplot = plt.imshow(img)
```



```
from PIL import Image
# ... your other code that loads the image into 'img'
img.thumbnail((64, 64), Image.Resampling.LANCZOS)
# resizes image in-place
plt.figure(figsize=(nSize, nSize))
sTitle='Resized'
plt.title(sTitle)
imgplot = plt.imshow(img)
plt.figure(figsize=(nSize, nSize))
sTitle='Resized with Bi-Cubic'
plt.title(sTitle)
imgplot = plt.imshow(img, interpolation="bicubic")
print('### Done!! ################################################')
```

OUTPUT :





PRACTICAL 12

AIM : To predict the price of any item using supervised learning algorithm.

CONCEPT :

Supervised Learning is a machine learning technique in which a model learns from previously labelled data to make predictions on new data. For predicting the price of an item, factors such as **size, quality, brand, location, or other features** are used as input. Regression algorithms such as **Linear Regression** can learn the relationship between these factors and the item's price. The trained model can then predict the price of a new item based on its given features.

CODE :

```
# Importing required libraries
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
# ------------------------------------------------------------
# Step 1: Load the California Housing dataset
# ------------------------------------------------------------
data = fetch_california_housing()
# Create a DataFrame containing the input features
X = pd.DataFrame(
    data.data,
    columns=data.feature_names
)
# Target variable
y = data.target
# Display dataset information
print("Dataset shape:", X.shape)
```

```
print("Features:", list(X.columns))
# ------------------------------------------------------------
# Step 2: Split dataset into training and testing data
# ------------------------------------------------------------
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
# Display number of training and testing samples
print("\nTraining samples:", X_train.shape[0])
print("Testing samples:", X_test.shape[0])
# ------------------------------------------------------------
# Step 3: Create and train Linear Regression model
# ------------------------------------------------------------
model = LinearRegression()
model.fit(
    X_train,
    y_train
)
# ------------------------------------------------------------
# Step 4: Make predictions on test data
# ------------------------------------------------------------
y_pred = model.predict(X_test)
# ------------------------------------------------------------
# Step 5: Evaluate the model
# ------------------------------------------------------------
mse = mean_squared_error(
```

```
    y_test,
  y_pred
)
r2 = r2_score(
    y_test,
    y_pred
)
print("\n========== Model Evaluation ==========")
print("Mean Squared Error (MSE):", mse)
print("R² Score:", r2)
# ------------------------------------------------------------
# Step 6: Display some actual vs predicted values
# ------------------------------------------------------------
comparison = pd.DataFrame({
    "Actual Price": y_test,
    "Predicted Price": y_pred
})
print("\n========== Actual vs Predicted ==========")
print(comparison.head(10))
# ------------------------------------------------------------
# Step 7: Plot Actual vs Predicted prices
# ------------------------------------------------------------
plt.figure(
    figsize=(8, 6)
)
# Create scatter plot
plt.scatter(
    y_test,
    y_pred,
```

```
    alpha=0.5
)
# Perfect prediction line
min_value = min(
    y_test.min(),
    y_pred.min()
)
max_value = max(
    y_test.max(),
    y_pred.max()
)
plt.plot(
    [min_value, max_value],
    [min_value, max_value],
    color="red",
    linewidth=2,
    label="Perfect Prediction"
)
# Add labels and title
plt.xlabel("Actual Price")
plt.ylabel("Predicted Price")
plt.title("Actual vs Predicted Median House Values")
# Display legend and grid
plt.legend()
plt.grid(True)
# Display the graph
plt.show()
```

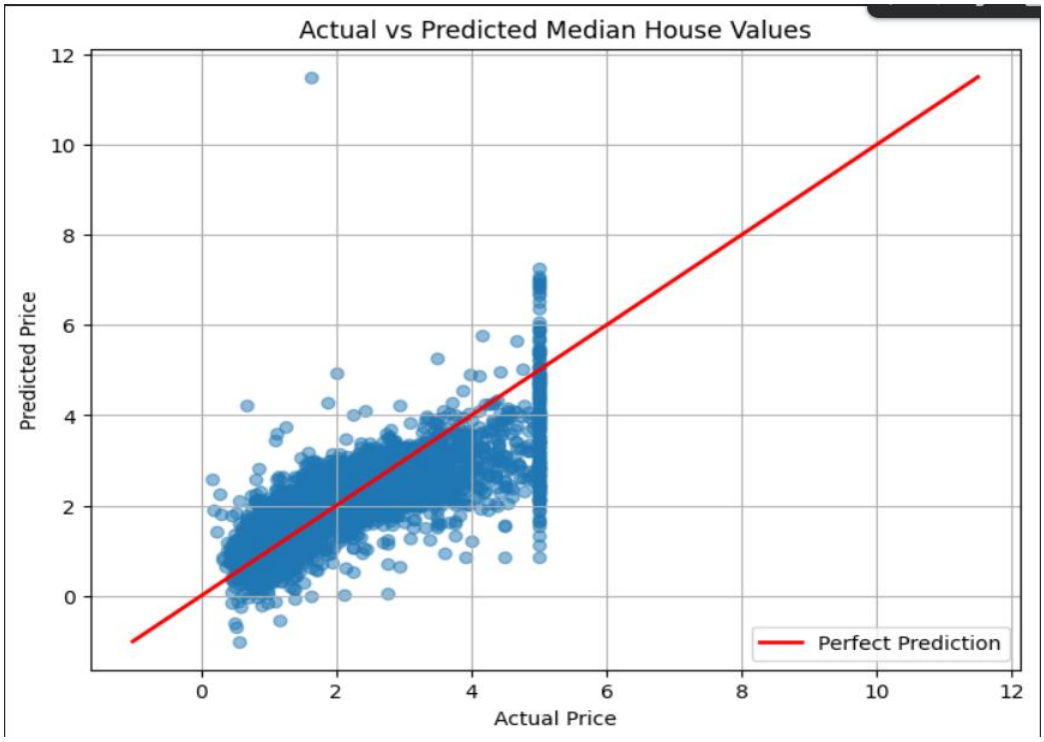
OUTPUT :

```
Dataset shape: (20640, 8)
Features: ['MedInc', 'HouseAge', 'AveRooms', 'AveBedrms', 'Population', 'AveOccup', 'Latitude', 'Longitude']

Training samples: 16512
Testing samples: 4128

========== Model Evaluation ==========
Mean Squared Error (MSE): 0.5558915986952444
R² Score: 0.5757877060324508

========== Actual vs Predicted ==========
   Actual Price  Predicted Price
0       0.47700         0.719123
1       0.45800         1.764017
2       5.00001         2.709659
3       2.18600         2.838926
4       2.78000         2.604657
5       1.58700         2.011754
6       1.98200         2.645500
7       1.57500         2.168755
8       3.40000         2.740746
9       4.46600         3.915615
```



PRACTICAL 13

AIM : To predict the label using an unsupervised learning algorithm.

CONCEPT :

Unsupervised learning is a machine learning technique that works with unlabeled data. It identifies hidden patterns or groups in the data without predefined labels. Clustering algorithms, such as K-Means, can group similar data points together and assign them to clusters. These clusters can then be used to predict or identify the label of new data based on similarity.

CODE :

```
# Step 1 — Install / Import Libraries

# These libraries come pre-installed in Google Colab,

# so there is no need to use !pip install.

import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

import seaborn as sns

from sklearn.datasets import load_iris

from sklearn.preprocessing import StandardScaler

from sklearn.decomposition import PCA

from sklearn.cluster import KMeans

from sklearn.metrics import (

    adjusted_rand_score,

    silhouette_score,

    confusion_matrix

)

# ============================================================

# Step 2 — Load the Dataset

# ============================================================

# Load the Iris dataset
```

```
iris = load_iris()


# Features (X) — 4 measurements of each flower
X = pd.DataFrame(
    iris.data,
    columns=iris.feature_names
)
# True labels (y) are kept ONLY for evaluation.
# The unsupervised algorithm will NOT use these during training.
y_true = iris.target
label_names = iris.target_names
print("Shape of feature data:", X.shape)
X.head()
# ============================================================
# Step 3 — Explore the Data
# ============================================================
# Display basic statistics
X.describe()
# Visualize how the real species differ
df_viz = X.copy()
df_viz['species'] = [
    label_names[i] for i in y_true
]
sns.pairplot(
    df_viz,
    hue='species'
)
plt.show()
# ============================================================
```

```
# Step 4 — Scale the Features
# ============================================================
# Clustering algorithms are sensitive to the scale of data.
# Standardization gives the features:
# Mean = 0
# Standard Deviation = 1
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
# ============================================================
# Step 5 — Find the Optimal Number of Clusters
#           Using the Elbow Method
# ============================================================
# In unsupervised learning, we normally don't know
# how many groups (K) exist.
inertia_values = []
k_range = range(1, 10)
for k in k_range:
    kmeans_temp = KMeans(
        n_clusters=k,
        random_state=42,
        n_init=10
    )
    kmeans_temp.fit(X_scaled)
    # Inertia = how tightly grouped the clusters are
    inertia_values.append(kmeans_temp.inertia_)
# Plot the Elbow Method
plt.figure(figsize=(8, 5))
plt.plot(
    k_range,
```

```
    inertia_values,
    marker='o'
)
plt.xlabel('Number of Clusters (K)')
plt.ylabel('Inertia')
plt.title('Elbow Method to Find Optimal K')
plt.show()
# For the Iris dataset, the elbow usually suggests K = 3.
# ============================================================
# Step 6 — Apply K-Means Clustering
# ============================================================
# Choose K = 3 based on the Elbow Method.
kmeans = KMeans(
    n_clusters=3,
    random_state=42,
    n_init=10
)
cluster_labels = kmeans.fit_predict(X_scaled)
# Add cluster assignments to the dataframe
X['predicted_cluster'] = cluster_labels
X['actual_species'] = [
    label_names[i] for i in y_true
]
X.head(10)
# ============================================================
# Step 7 — Map Cluster Numbers to Real Label Names
# ============================================================
# K-Means gives arbitrary cluster numbers such as
# 0, 1 and 2.
```

```
#
# These numbers do not automatically match the
# actual species labels.
mapping = {}
for cluster in np.unique(cluster_labels):
    # Find the data points belonging to this cluster
    mask = (cluster_labels == cluster)
    # Find the most common true label in the cluster
    most_common_label = pd.Series(
        y_true[mask]
    ).mode()[0]
    mapping[cluster] = most_common_label
# Create the final predicted label column
predicted_labels = np.array([
    mapping[c] for c in cluster_labels
])
X['predicted_label_name'] = [
    label_names[i] for i in predicted_labels
]
# Display actual and predicted labels
X[
    [
        'actual_species',
        'predicted_cluster',
        'predicted_label_name'
    ]
].head(10)
# ============================================================
# Step 8 — Evaluate the Clustering Results
```

```
# ============================================================


# Calculate accuracy
accuracy = (
    predicted_labels == y_true
).mean()
print(
    f"Clustering-based label prediction accuracy: "
    f"{accuracy * 100:.2f}%"
)
# Adjusted Rand Index (ARI)
# 1.0 = perfect agreement
ari = adjusted_rand_score(
    y_true,
    cluster_labels
)
print(
    f"Adjusted Rand Index: {ari:.3f}"
)
# Silhouette Score
# Closer to 1 = better-separated clusters
sil_score = silhouette_score(
    X_scaled,
    cluster_labels
)
print(
    f"Silhouette Score: {sil_score:.3f}"
)
```

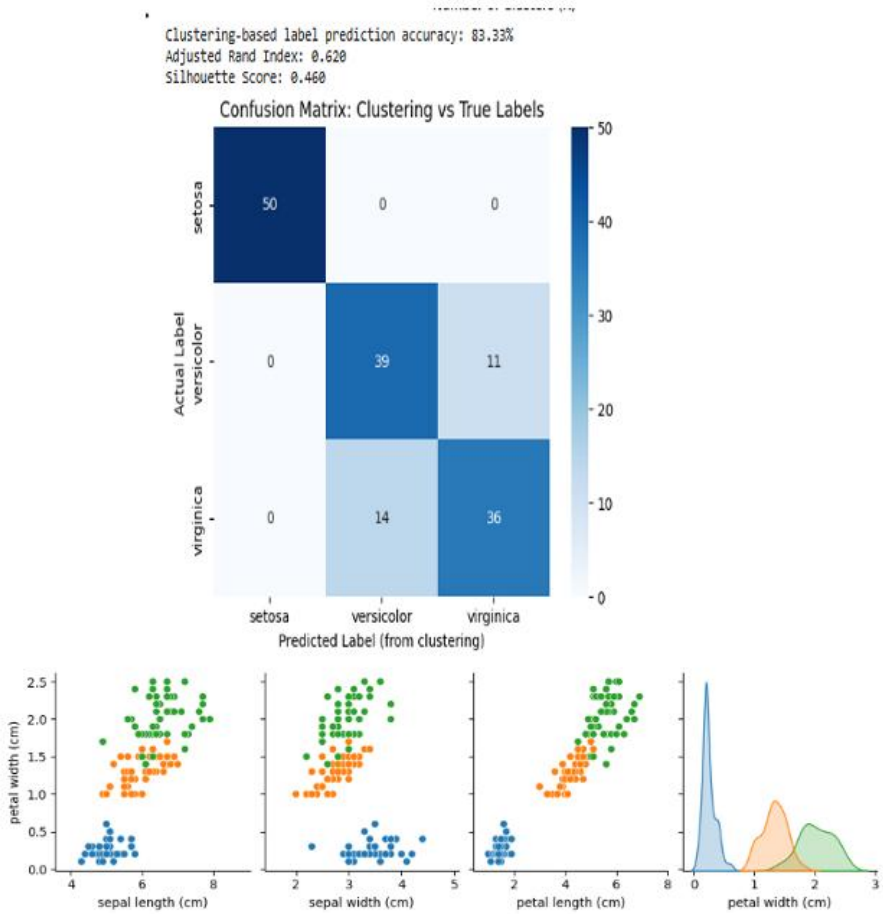
```
# ============================================================
# Step 9 — Confusion Matrix
# ============================================================
# Compare actual labels with predicted labels
cm = confusion_matrix(
    y_true,
    predicted_labels
)
plt.figure(figsize=(6, 5))
sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap='Blues',
    xticklabels=label_names,
    yticklabels=label_names
)
plt.xlabel('Predicted Label (from clustering)')
plt.ylabel('Actual Label')
plt.title('Confusion Matrix: Clustering vs True Labels')
plt.show()
# ============================================================
# Step 10 — Visualize Clusters in 2D using PCA
# ============================================================
# The Iris dataset has 4 features.
# PCA reduces the data to 2 dimensions
# for easy visualization.
pca = PCA(
```

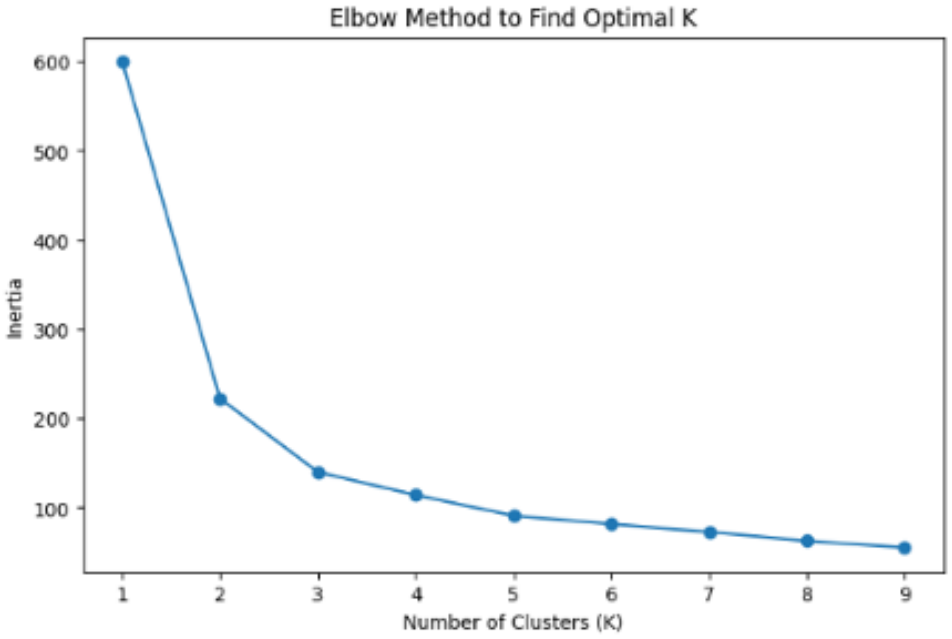
```
    n_components=2)

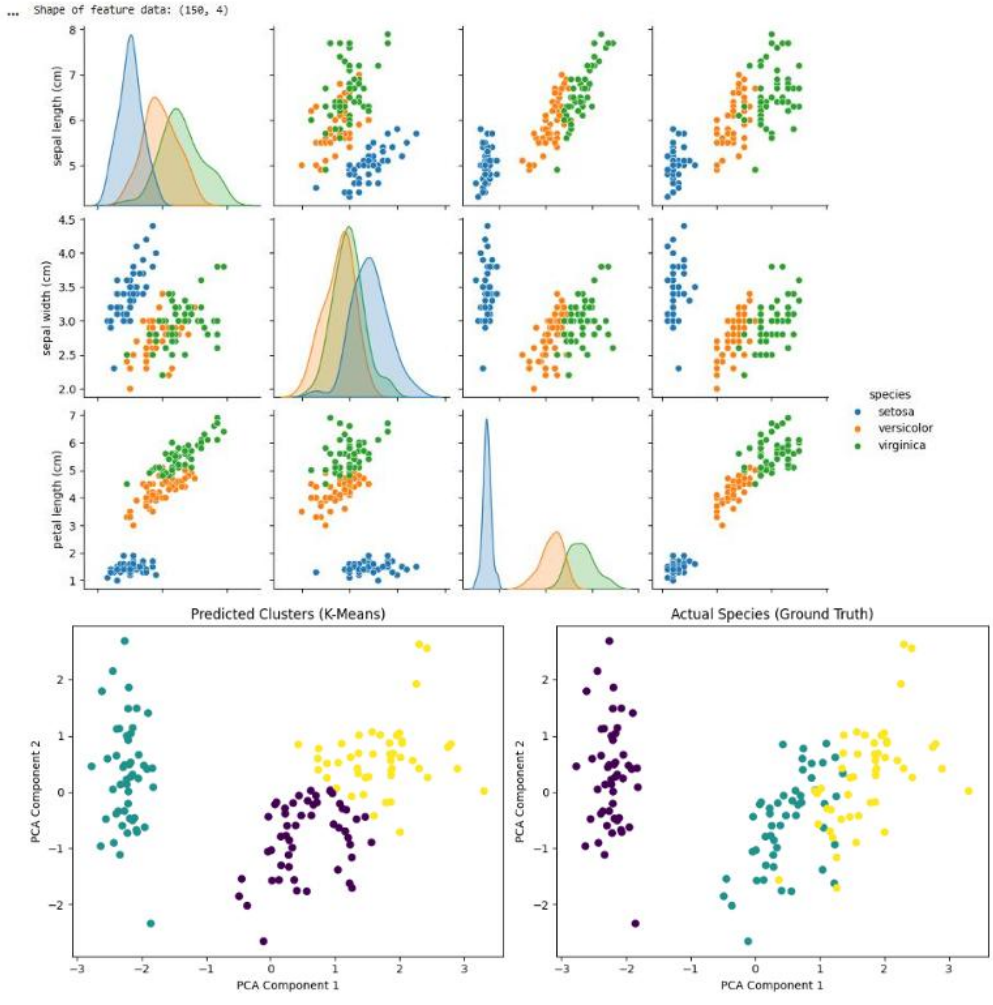
X_pca = pca.fit_transform(X_scaled)
plt.figure(figsize=(12, 5))
# ------------------------------------------------------------
# Left Plot — Predicted Clusters
# ------------------------------------------------------------
plt.subplot(1, 2, 1)
plt.scatter(
    X_pca[:, 0],
    X_pca[:, 1],
    c=cluster_labels,
    cmap='viridis'
)
plt.title('Predicted Clusters (K-Means)')
plt.xlabel('PCA Component 1')
plt.ylabel('PCA Component 2')
# ------------------------------------------------------------
# Right Plot — Actual Species
# ------------------------------------------------------------
plt.subplot(1, 2, 2)
plt.scatter(
    X_pca[:, 0],
    X_pca[:, 1],
    c=y_true,
    cmap='viridis'
)
plt.title('Actual Species (Ground Truth)')
plt.xlabel('PCA Component 1')
```

```
plt.ylabel('PCA Component 2')
plt.tight_layout()
plt.show()
```

OUTPUT :




Elbow Method to Find Optimal K
Inertia
Number of Clusters (K)


Shape of feature data: (150, 4)
sepal length (cm)
sepal width (cm)
petal length (cm)
species
setosa
versicolor
virginica
Predicted Clusters (K-Means)
Actual Species (Ground Truth)
PCA Component 1
PCA Component 2

PRACTICAL 14

AIM : To find the correlation between the two datasets.

CONCEPT :

Correlation is a statistical technique used to measure the relationship between two datasets. It shows how changes in one dataset are related to changes in the other. The correlation value ranges from **-1 to +1**, indicating negative, no, or positive correlation. It helps in understanding the strength and direction of the relationship between variables.

CODE :

```
import pandas as pd
import matplotlib.pyplot as plt


# Step 1: Read the two files
math = pd.read_excel("/content/students_math.csv")
science = pd.read_excel("/content/student_science.csv")


# Step 2: Display the datasets
print("Math Dataset:")
display(math)
print("Science Dataset:")
display(science)


# Step 3: Merge the two datasets using Student_ID
data = pd.merge(math, science, on="Student_ID")
print("Merged Dataset:")
display(data)


# Step 4: Find correlation between Math and Science
correlation = data["Math"].corr(data["Science"])
```

```
print("Correlation between Math and Science =", round(correlation, 2))


# Step 5: Display correlation matrix
print("\nCorrelation Matrix:")
display(data[["Math", "Science"]].corr())
# Step 6: Draw scatter plot
plt.scatter(data["Math"], data["Science"])
plt.xlabel("Math Marks")
plt.ylabel("Science Marks")
plt.title("Correlation between Math and Science Marks")
plt.show()
```

OUTPUT :

```
Correlation between Math and Science = 0.98
            Math   Science
Math       1.000000  0.976336
Science    0.976336  1.000000
```